University of Ain Temouchent - Department of Computer Science
Field: Computer Science
Module: Natural Language Processing

Practical Session 5 : TF-IDF For Movie Recommendation System

Supervisor: Dr. BENDIABDALLAH M.H
hakim.bendiabdallah@gmail.com /
hakim.bendiabdallah@univ-temouchent.edu.dz

Objective

The main objective of this third practical session (TP 5) is to build a simple movie recommendation system using content -based filtering . We will use natural language processing (NLP) techniques to:

- Process and transform movie metadata (genres, keywords, etc.).
- Compute similarity between movies.
- Provide movie recommendations based on a given input title.

Requirements

- Python 3.x and typical data science libraries (NumPy, pandas, scikit-learn, Matplotlib).
- A dataset containing movie information (e.g., `tmdb_5000_movies.csv`).
- Basic familiarity with JSON parsing and TF-IDF vectorization.

Data and Overview

We will use the `tmdb_5000_movies.csv` dataset, which provides movie information in JSON format for columns such as `genres` and `keywords`. The goal is to extract relevant text from these columns, transform it using TF-IDF, and then compute similarities among movies to generate recommendations.

Dr. BENDIABDALLAH M.H

Step-by-Step Instructions

Step 1: Load and Explore the Data

1. Import the necessary libraries.
2. Load the CSV file into a pandas DataFrame.
3. Inspect the columns and a few sample rows.

```
import pandas as pd
import matplotlib.pyplot as plt
import json

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity,
    euclidean_distances

df = pd.read_csv('tmdb_5000_movies.csv')
df.head()
```

Step 2: Understand the JSON Columns

1. The `genres` and `keywords` columns are stored as JSON strings.
2. Extract the `name` fields from each JSON entry and create a simple text string.

```
x = df.iloc[0]
x
```

```
print(x['genres'])
```

```
print(x['keywords'])
```

```
# Example of parsing the JSON string
j = json.loads(x['genres'])
j # This is now a Python list of dictionaries
```

```
# Example of joining names
'␣'.join(''.join(jj['name'].split()) for jj in j)
```

Dr. BENDIABDALLAH M.H

Step 3: Preprocessing and Creating a String Representation

1. Define a function `genres_and_keywords_to_string` that extracts genres and keywords from each row.
2. Convert the relevant data for each movie into a single string to be ingested by TfidfVectorizer.
3. Store the result in a new column `df['string']`.

```
def genres_and_keywords_to_string(row):
    genres = json.loads(row['genres'])
    genres = ' '.join(''.join(j['name'].split()) for j in genres)

    keywords = json.loads(row['keywords'])
    keywords = ' '.join(''.join(j['name'].split()) for j in keywords)

    return "%s %s" % (genres, keywords)
```

create a new string representation of each movie

```
df['string'] = df.apply(genres_and_keywords_to_string, axis=1)
df['string']
```

Step 4: Vectorization with TF-IDF

1. Create a `TfidfVectorizer`.
2. Fit and transform the `df['string']` data.
3. Obtain the matrix representation `X`.

```
tfidf = TfidfVectorizer(max_features=2000)
X = tfidf.fit_transform(df['string'])
X
```

Step 5: Compute Similarities

1. Build a mapping of `movie2idx` for title-based lookups.
2. Given a movie title, obtain its TF-IDF vector and compute cosine similarities between that vector and all others in `X`.

Dr. BENDIABDALLAH M.H

generate a mapping from movie title -> index (in df)

```
movie2idx = pd.Series(df.index, index=df['title'])
movie2idx
```

```
idx = movie2idx['Scream 3']
query = X[idx]
query
```

Compute similarity between query and every vector in X

```
scores = cosine_similarity(query, X)
scores = scores.flatten()
```

```
plt.plot(scores)
plt.show()
```

Get top 5 matches

```
recommended_idx = (-scores).argsort()[1:6]
df['title'].iloc[recommended_idx]
```

Step 6: Create a Recommendation Function

1. Wrap the logic in a `recommend` function that takes a movie title.
2. Looks up the index, computes similarities, and returns top N matching titles with default N=5.

```
# define your fonction here
```

Test the recommendation function

Recommendations for 'Scream 3'

```
print("Recommendations for 'Scream 3':")
print(recommend('Scream 3',3))
```

Try with these movie titles:

- Iron Man 2
- Inception
- Harry Potter and the Goblet of Fire

Dr. BENDIABDALLAH M.H

- The Hobbit: The Desolation of Smaug
- Kung Fu Panda
- The Fast and the Furious

Conclusion

In this TP, you learned how to build a simple movie recommendation system using:

- Basic data parsing and cleaning,
- TF-IDF vectorization to represent movies by their textual metadata,
- Cosine similarity to measure closeness between movies,
- A modular recommendation function to suggest similar movies.

Feel free to experiment with different features (e.g., adding production companies, cast, or using movie plots) and different similarity measures (e.g., `euclidean_distances`) to improve your recommendation results.

Dr.BENDIABDALLAH M.H